

can use the `copy.deepcopy()` function to make copies of the object (and copies of any objects it references).

You shouldn't use mutable objects in `def` statements for default arguments, because they're created once when the `def` statement is run rather than each time the function is called. A better idea is to make the default argument `None`, and then add code that checks for `None` and creates a mutable object when the function is called.

A subtle gotcha is the string concatenation of several smaller strings with the `+` operator in a loop. For small numbers of iteration, this syntax is fine. But under the hood, Python is constantly creating and destroying string objects on each iteration. A better approach is to append the smaller strings into a list and then call the `join()` operator to create the final string.

The `sort()` method sorts by numeric code points, which isn't the same as alphabetical order: uppercase Z is sorted before lowercase a. To fix this issue, you can call `sort(key=str.lower)`.

Floating-point numbers have slight rounding errors as a side effect of how they represent numbers. For most programs, this isn't important. But if it does matter for your program, you can use Python's `decimal` module.

Never chain together `!=` operators, because expressions like `'cat' != 'dog' != 'cat'` will, confusingly, evaluate to `True`.

Although this chapter described the Python gotchas that you're most likely to encounter, they don't occur daily in most real-world code. Python does a great job of minimizing the surprises you might find in your programs. In the next chapter, we'll cover some gotchas that are even rarer and downright bizarre. It's almost impossible that you'll ever encounter these Python language oddities if you aren't searching for them, but it'll be fun to explore the reasons they exist.